

INTERN OPERATING GUIDE

Edward Agent Stack

A practical setup guide for interns using Codex with Edward's default engineering rules: ask Codex first, use gstack when stuck, keep project facts written down, and escalate with options.

Generated with Kami · 2026-04-30

Source: [giaphutran12/edward-agent-stack](#)

Secrets rule: never inspect, print, diff, or summarize local real env values.

| Contents

01 What This Repo Is

02 Fresh Mac Install

03 Tool Stack

04 Auth Gates And Secrets

05 Daily Workflow

06 GStack, Repowise, Escalation

01 · What This Repo Is

This repo gives a fresh Codex session the standard setup and rules for intern engineering work.

Edward Rules

Stable defaults: ask Codex first, back claims with sources or tests, prefer Postgres/Supabase before adding infra, and do not make destructive changes casually.

Project Notes

Current facts for one project: goal, stack, priority, gotchas, and what must be approved by Edward.

Decision Notes

Dated files that explain why a choice was made. If Edward decides something reusable, Codex writes it down so interns do not repeat the same debate.

Boundaries

- It does not replace project-specific onboarding. Each project still needs a `PROJECT.md`.
- It does not make shared API keys safe. Access control still needs scoped accounts, logs, revocation, and rotation.
- It does not remove judgment. Interns still need to understand Codex's options before choosing one.

02 · Fresh Mac Install

Paste the block below into Codex on a fresh Mac. The intern should not hand-install random tools first. Codex runs the scripts, fixes safe missing dependencies, and reports blocked auth/API gates.

Install Edward's agent stack on this fresh MacBook. Clone <https://github.com/giaphutran12/edward-agent-stack> to `~/edward-agent-stack`, read `AGENTS.md`, then run `./scripts/bootstrap-macos.sh`, `./scripts/install.sh`, `./scripts/verify.sh`, and `./scripts/auth-doctor.sh`. Use `/caveman ultra` for terse token-saving communication. Install prerequisites automatically when safe: Xcode Command Line Tools prompt, Homebrew, Node/npm/npx, Bun, Python/pip/uv, GitHub CLI, ripgrep, jq, tmux, ffmpeg, Supabase CLI, Vercel CLI, Claude Code, Nia CLI, Repowise, Caveman, Edward skills, and giaphutran12/codex-gstack for Codex using `./setup --host codex`. Do not inspect or print local `real env/secret` files. If a tool needs login, API key, license acceptance, or a GUI first-run step, stop and tell me exactly what is missing instead of forcing it.

1. `./scripts/bootstrap-macos.sh`: Xcode Command Line Tools, Homebrew, shell path, Rosetta when needed.
2. `./scripts/install.sh`: core CLIs, skills, Edward gstack fork, MCP template.
3. `./scripts/verify.sh`: binary/file sanity check.
4. `./scripts/auth-doctor.sh`: safe auth report without printing secrets.

03 · Tool Stack

Layer	Default	Why
Primary agent	Codex	Interns ask Codex first with full repo/task context.
Workflow	Caveman, Edward Rules, gstack	Short communication, stable defaults, investigate/ship paths.
Repo context	Repowise, ripgrep	Orient before editing; search fast.
Memory/ research	MemPalace, Nia, Exa	Durable agent memory, indexed context, current web research.
Shipping	GitHub, Vercel, Supabase CLI	PRs, deploy inspection, migrations/project management.
Runtime basics	Docker, Python/uv, Bun/Node, jq, tmux	Local services, scripts, JS/TS apps, JSON, long sessions.

Not default Notion, TinyFish, OMX, Kiro, `mlx_whisper`, shared-secret vaults.

04 · Auth Gates And Secrets

Installer can install binaries. It cannot create accounts, approve OAuth, accept licenses, or invent API keys.

Tool	Command / Gate	Note
Codex	<code>codex login</code>	Primary coding agent.
GitHub	<code>gh auth login --web</code>	Clone, push, PRs, issues.
Vercel	<code>vercel login</code>	Deploy and project inspection.
Supabase	<code>supabase login</code>	Migrations and project management.
Nia	<code>nia auth login</code>	Indexed repo/docs/context search.
Exa	<code>local EXA_API_KEY</code>	Keep in local Codex config only.
Repowise Gemini	<code>.repowise/.env</code>	Local ignored project file. Never print it.

Shared-secret vaults are project-specific, not default. Once a credential is readable, the useful controls are scope, logs, revocation, and rotation. For interns, use per-user accounts when possible, scoped/staging keys, ignored local env files, logs, revocation, and rotation.

05 · Daily Workflow

1. Load `/caveman ultra`.
2. Load `$edward-rules`.
3. Read the project `PROJECT.md`, recent decision notes, and `AGENTS.local.md` if present.
4. Ask Codex first. Understand the options and blast radius before escalating.
5. Before PR/end of meaningful work, run decision capture and update project notes if context changed.

Default Edward Question

Did you ask Codex? What did Codex say?

06 · GStack, Repowise, Escalation

GStack

```
git clone --single-branch --depth 1 https://github.com/giaphutran12/codex-gstack.git
~/.gstack/repos/gstack
cd ~/.gstack/repos/gstack
./setup --host codex
```

Use `gstack investigate` before saying stuck. Use `gstack ship` for PR/review/shipping flow.

Repowise

```
./scripts/repowise-update.sh /path/to/project --provider gemini
```

Repowise is for repo/code orientation, not Edward Rules. Use it before broad repo edits so Codex starts from current codebase structure instead of guessing.

Escalation Template

Edward, need decision.

Question:

Context:

What Codex said:

Options:

A)

B)

C)

Tradeoff:

Risk/blast radius:

What I tested:

My recommendation:

Send the question, context, what Codex said, options, tradeoff, risk, test evidence, and your recommendation. Include why repo/docs/internet/Codex cannot answer it.